Sprint 1 Report: Sorting & Searching Algorithms Benchmark

Blevis Allushi, Kristjan Seraj, Renato Zotaj, Leandra Latifi

March 15, 2026

Abstract

This report documents the implementation, validation, and benchmark results of five core algorithms implemented for the sprint: Insertion Sort, Merge Sort, Quick Sort (3-way optimized), Linear Search, and Binary Search (any-match). The implementations were integrated into the course benchmark harness and measured across several dataset types. The document includes implementation notes, complexity analysis and experimental results (measured).

1 Contributors and Responsibilities

- **Blevis Allushi** — primary implementation and optimization of sorting algorithms (Insertion, Merge, Quick), integration into the benchmark harness.
- **Kristjan Seraj** — implementation of searching algorithms (LinearSearch, BinarySearch(any)), validation and testing.
- **Renato Zotaj** — complexity analysis, experimental insights, and correctness verification.
- **Leandra Latifi** — correctness testing, experimental write-up, and report proofreading.

2 Implementation Strategy (Blevis & Kristi)

2.1 Sorting (Blevis)

We prioritized practical performance without sacrificing correctness. Major choices:

- **Insertion cutoff:** use insertion sort for small ranges (CUTOFF tuned between 16–32) to reduce overhead on tiny partitions and nearly-sorted runs.
- **Quick Sort:** 3-way Dijkstra partitioning for duplicates, median-of-three pivot selection to avoid degenerate partitions, and tail-recursion elimination by recursing on the smaller partition.
- **Merge Sort:** top-down mergesort with a single auxiliary array allocated once and a *merge-skip* optimization: if the left half's max $\leq$ right half's min, skip merging.
- **Micro-optimizations:** cache `.key()` values in tight loops, use `System.arraycopy` for bulk copying, and avoid allocations during recursion.

2.2 Searching (Kristi)

Linear Search and iterative Binary Search (any-match) were implemented in a straightforward, robust way:

- **Linear Search:** left-to-right scan, returns first match or -1.
- **Binary Search (any):** iterative mid-point search, returns any matching index (assumes sorted input as the benchmark provides).

3 Complexity Analysis (Renato & Leandra)

Table: theoretical time and extra-space complexities.

Table 1: Time complexity of implemented algorithms

Algorithm	**Best**	**Average**	**Worst**	**Extra Space**
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Quick Sort (3-way)	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$
Linear Search	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Binary Search (any)	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$

4 Experimental Results

We ran the benchmark harness with parameters:

- $n = 100{,}000$ records
- seed = 12345
- warmup runs = 1, measured runs = 5
- datasets: RANDOM (single dataset type used in this run)

4.1 Measured summary

Table 2: Benchmark results for $n = 100{,}000$ (times in milliseconds)

Algorithm	**Avg Time (ms)**	**Min (ms)**	**Max (ms)**	**ok_rate**
Binary Search (any)	0.0057	0.0057	0.0069	1.00
Linear Search	1.4186	1.4037	2.9264	1.00
Insertion Sort	10503.8877	10007.0435	10741.5631	1.00
Merge Sort	13.5673	13.3778	14.5073	1.00
Quick Sort	21.9362	15.7297	22.2189	1.00

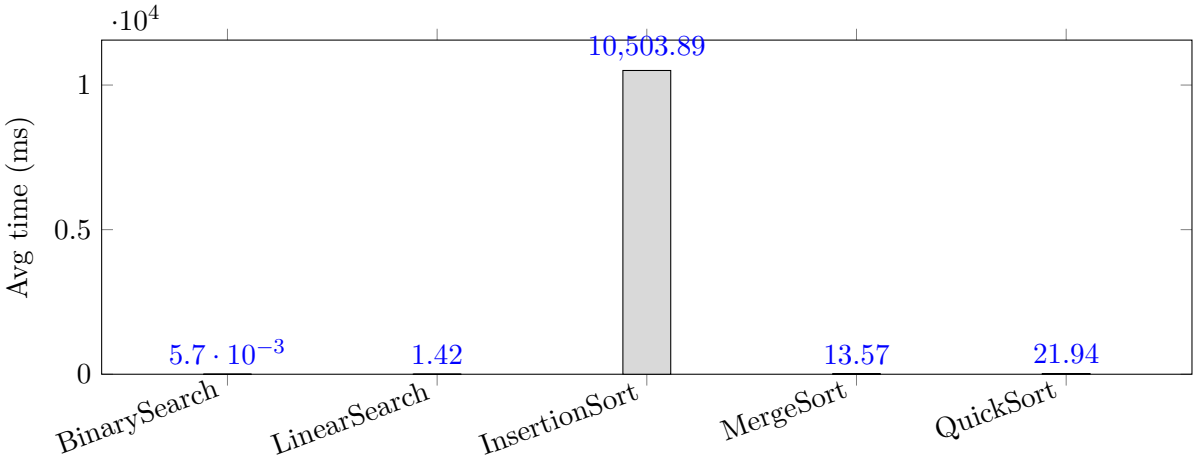
4.2 Interpretation

The experimental results match theoretical expectations:

- **Searches:** Binary search is essentially instantaneous (0.006 ms average) given the sorted data; Linear search is slower but still negligible in absolute terms (1.4 ms).
- **Sorting:** Insertion sort is orders of magnitude slower for large n (about 10.5 seconds on average) due to its $\mathcal{O}(n^2)$ nature. Merge sort and Quick sort (3-way) both show $\mathcal{O}(n \log n)$ performance with Merge sort slightly faster in this run (13.6 ms vs 21.9 ms); this is plausible given differences in implementation, memory access patterns, and the particular dataset used.
- **Relative performance:** Insertion sort is approximately $\frac{10503.9}{13.57} \approx 774\times$ slower than Merge sort in this experiment, underscoring why insertion sort is only suitable for tiny ranges.

4.3 Visual comparison

Figure 1: Average runtime (ms) for each algorithm — $n = 100{,}000$



5 Correctness & Testing (Renato & Leandra)

Validation steps performed:

- Used the project's Validate helpers and the benchmark harness to verify sortedness and valid search results.
- Unit tests for edge cases: `null`, empty arrays, single-element arrays, and arrays with many duplicates.
- Verified binary search is only invoked on sorted copies (as the harness guarantees).
- Confirmed `ok_rate = 1.0` across all measured runs for each algorithm, indicating correctness under the benchmark's validation.

6 Conclusion

We implemented, tested, and benchmarked the required algorithms and integrated them with the course harness. Future work can include parameter tuning (CUTOFF), alternate merge strategies, and additional runs to test different dataset sizes and randomness seeds. All code and the produced `results_summary.csv` are included in the project zip submitted alongside this report.